

# Day 2 Lab Sheet: Install Tools, Use the Terminal, Make the Notes Board Work

---

**Goal:** By the end of this lab you will have Node.js and Git installed, you will have used the terminal, and your Notes Board will add and delete notes in the browser.

**Time:** about 100 minutes, plus the quiz.

**What you need:** The `Workshop-Installers` folder from your Day 1 homework, and your `Workshop` folder with the Day 1 result.

If your Day 1 files are missing or broken, copy the **Day 2 starter** folder from your instructor. It has everything finished up to Day 1. The **done** folder is the finished result for today.

---

## Part 1: Install Node.js (10 minutes)

---

1. Open your `Workshop-Installers` folder and find the **Node.js 24 LTS** installer. It is a file that ends in `.msi`.
2. **Double-click** it. If Windows asks "Do you want to allow this app to make changes?", click **Yes**.
3. In the installer, click **Next** on each screen and keep the default choices. Accept the licence agreement when asked.
4. If you see a page about **"Tools for Native Modules"**, do **not** tick the box.
5. Click **Install**, wait, and click **Finish**.

**Do not open VS Code yet.** If VS Code was already open, you will close it in Part 3.

---

## Part 2: Install Git (10 minutes)

---

1. In your `Workshop-Installers` folder, find the **Git for Windows** installer. It is a file that ends in `.exe`.
  2. **Double-click** it. Click **Yes** if Windows asks for permission.
  3. Click **Next** on each screen and keep the default choices.
  4. If you see a screen called **"Choosing the default editor used by Git"**:
    - Choose **Visual Studio Code** if it is in the list.
    - Otherwise choose **Notepad**.
    - Do **not** choose Vim. It is hard for beginners to close.
  5. Click **Install**, wait, and click **Finish**.
-

## Part 3: Restart VS Code and open the terminal (5 minutes)

---

A terminal learns about new programs only when it starts. So we restart VS Code first.

1. **Close VS Code completely** (click the X on the window).
2. Open VS Code again. It should open your `Workshop` folder. If not, click **File**, then **Open Recent**.
3. Click **Terminal** in the top menu, then **New Terminal**.
4. In the terminal panel, click the small **down arrow** next to the **+** icon, and choose **Command Prompt**.

**Check:** At the bottom you see a prompt like `C:\Users\YourName\Documents\Workshop>` .

---

## Part 4: Check your tools (5 minutes)

---

Type each command and press **Enter**. After each one, compare what you see.

```
node -v
```

You should see a version that starts with `v24` , for example `v24.x.x` .

```
npm -v
```

You should see a version number.

```
git --version
```

You should see something like `git version 2.x.x` .

**If you see "is not recognized as an internal or external command":** close VS Code, open it again, open a new terminal, and try again. If it still fails, see the Stuck Sheet.

### Run the setup check

1. Find the file `setup-check.js` in the installer kit folder (your instructor will tell you where it is on the LAN share or USB).
2. Copy it into your `Workshop` folder.
3. In the terminal, type:

```
node setup-check.js
```

**Check:** All four lines show **PASS**. Do not move on until they do. Raise your hand if a line shows **FAIL**.

---

## Part 5: Practice with the terminal (10 minutes)

---

Type these commands one by one. Read what each one prints.

```
cd
```

This shows where you are.

```
dir
```

This lists the files and folders. You should see `notes-board` .

```
mkdir practice  
cd practice
```

You made a new folder and went inside it. Look at the prompt. It changed.

```
echo Hello from the terminal > hello.txt  
type hello.txt
```

The first line wrote a file. The second line showed what is inside it.

```
cd ..
```

You went one folder up. Look at the prompt again.

Now try these useful tricks:

- Type `cd note` and press **Tab**. The terminal completes the name `notes-board` .
- Press the **Up arrow**. Your last command comes back.
- Type `cls` and press Enter. The screen clears.

**Check:** You can move into a folder and out of it, and you can explain what the prompt shows.

---

## Part 6: Try JavaScript in the browser console (10 minutes)

---

1. Open any web page in your browser. Press **F12**, and click the **Console** tab.
2. Type each line below and press **Enter**. After each line, look at the result.

```
console.log("Hello, world");
```

```
const college = "Mandsaur University";  
college
```

```
let count = 0;  
count = count + 1;  
count
```

```
const days = ["Mon", "Tue", "Wed"];  
days.push("Thu");  
days.length
```

```
const note = { id: 1, title: "Welcome", text: "Hello" };  
note.title
```

```
function greet(name) {  
  return "Hello, " + name;  
}  
greet("your name")
```

**Check:** You saw a result after each line, and you can say what a variable, an array, an object and a function are.

Close the tools with **F12**.

---

## Part 7: Make the Notes Board work (35 minutes)

---

We now connect JavaScript to the page. Work in your `notes-board` folder.

### 7a. Update `index.html`

Open `index.html` and make five small changes.

**Change 1.** Under the `<textarea>` line, add a character counter:

```
<p class="counter"><span id="char-count">0</span> / 300 characters</p>
```

**Change 2.** Under the `<button>` line, add a place for messages:

```
<p id="form-message" class="message" role="status"></p>
```

**Change 3.** Change the heading `All notes` to show a number:

```
<h2>All notes <span id="note-count" class="badge"></span></h2>
```

**Change 4.** Delete the three sample notes. Replace the whole `<div id="notes-list">` block with an empty one. JavaScript will fill it:

```
<div id="notes-list"></div>
```

**Change 5.** At the very end of `<body>` , just before `</body>` , add the script tag:

```
<script src="js/app.js"></script>
```

## 7b. Create `js/app.js`

1. Click the `notes-board` folder, click **New Folder**, and name it `js` .
2. Click the `js` folder, click **New File**, and name it `app.js` .
3. Type the code below, **one section at a time**. Your instructor will explain each section before you type it.

## Section 1: The data

```
// app.js (Day 2)
//
// The Notes Board now works in the browser.
// Our notes live in a JavaScript array. When you refresh the page,
// the array is created again and your new notes are lost.
// On Days 3 and 4 we fix this with a server and a database.

// 1. Data: a list (array) of notes. Each note is an object.
const notes = [
  { id: 1, title: "Welcome", text: "This is a sample note. Try adding your own." },
  { id: 2, title: "Study plan", text: "Learn HTML on Monday, CSS on Tuesday, and
JavaScript on Wednesday." }
];

let nextId = 3;
```

## Section 2: Find the parts of the page

```
// 2. Find the parts of the page we need to change.
const form = document.getElementById("note-form");
const titleInput = document.getElementById("note-title");
const textInput = document.getElementById("note-text");
const charCount = document.getElementById("char-count");
const message = document.getElementById("form-message");
const notesList = document.getElementById("notes-list");
const noteCount = document.getElementById("note-count");
```

## Section 3: Show a message

```
// 3. Show a short message under the form.
function showMessage(text, isError) {
  message.textContent = text;
  message.className = isError ? "message error" : "message ok";
}
```

## Section 4: Draw the notes

```
// 4. Draw all notes on the page.
function renderNotes() {
  // Remove what is on the page now, then draw again from the array.
  notesList.textContent = "";

  if (notes.length === 0) {
    const empty = document.createElement("p");
    empty.className = "empty";
    empty.textContent = "No notes yet. Add your first note above.";
    notesList.appendChild(empty);
  }

  for (const note of notes) {
    const article = document.createElement("article");
    article.className = "note";

    const heading = document.createElement("h3");
    heading.textContent = note.title;

    const body = document.createElement("p");
    body.textContent = note.text;

    const deleteButton = document.createElement("button");
    deleteButton.type = "button";
    deleteButton.className = "delete-button";
    deleteButton.textContent = "Delete";
    deleteButton.addEventListener("click", function () {
      deleteNote(note.id);
    });

    article.appendChild(heading);
    article.appendChild(body);
    article.appendChild(deleteButton);
    notesList.appendChild(article);
  }

  noteCount.textContent = notes.length;
}
```

## Section 5 and 6: Add and delete

```
// 5. Add a note to the array (newest note first).
function addNote(title, text) {
  const note = { id: nextId, title: title, text: text };
  nextId = nextId + 1;
  notes.unshift(note);
  renderNotes();
}

// 6. Remove a note from the array.
function deleteNote(id) {
  const index = notes.findIndex(function (note) {
    return note.id === id;
  });

  if (index !== -1) {
    notes.splice(index, 1);
  }

  renderNotes();
}
```

## Section 7, 8 and 9: Events and the first draw

```
// 7. When the form is submitted, check the input and add the note.
form.addEventListener("submit", function (event) {
  // Stop the browser from reloading the page.
  event.preventDefault();

  const title = titleInput.value.trim();
  const text = textInput.value.trim();

  if (title === "" || text === "") {
    showMessage("Please write both a title and a note.", true);
    return;
  }

  addNote(title, text);
  form.reset();
  charCount.textContent = "0";
  showMessage("Note added.", false);
  titleInput.focus();
});

// 8. Show how many characters are used while the user types.
textInput.addEventListener("input", function () {
  charCount.textContent = textInput.value.length;
});

// 9. Draw the notes once when the page opens.
renderNotes();
```

### 7c. Add the new styles

Our new elements need style. Open the file `Day-2-Tools-and-JavaScript\done\notes-board\css\style.css`. Copy everything **after** the line `/* ----- Added on Day 2 ----- */` and paste it at the **end** of your `css\style.css`. Save.

### 7d. Test it

1. Open `index.html` in the browser (double-click it, or press **F5** if it is already open).
2. Press **F12** and look at the **Console** tab. It should show **no red errors**.
3. Try these tests:
  - Add a note with a title and text. It appears at the top.

- Press **Add note** with empty boxes. You see an error message.
- Click **Delete** on a note. It disappears, and the number changes.
- Type some text in the note box. The counter changes.
- Add a note with this text: `<b>bold</b>` . It shows as plain letters. This is the safe `textContent` working.
- Refresh the page with **F5**. All your new notes are gone. This is what we will fix on Days 3 and 4.

**Check:** Add, delete, error message and counter all work. If something fails, read the **Console** for a red message, and see the Stuck Sheet.

---

## Part 8: Make the About page use JavaScript (15 minutes)

---

Today the details on your About page are typed into the HTML. We now keep them in one JavaScript file, so you only edit one small file.

### 8a. Create `js/student-details.js`

Create the file `js\student-details.js` and type this. Replace each placeholder in angle brackets (for example `<enter-your-name>` ) with **your own details**. **You decide what to show**. Delete the text between the quotes ( `" "` ) to hide a line.

```
// student-details.js
//
// This is the ONLY file you need to edit to change the About page.
//
// How to use it:
// 1. Replace each <enter-...> placeholder with your own details.
// 2. You decide what to show. Delete the text between the quotes ("") to hide a
line.
// 3. Save the file and refresh the About page.
//
// We encourage you to keep the instructor details at the bottom, so that
// your visitors know where this project was learned.

const studentDetails = {
  name: "<enter-your-name>", // Example: "Asha Sharma"
  rollNumber: "<enter-your-roll-number>", // Example: "BCA2024-017"
  className: "BCA Second Year",
  batch: "2025-26",
  college: "Mandsaur University",
  githubProfile: "<enter-your-github-profile-link>", // Example:
"https://github.com/your-username"
  tagline: "<enter-your-tagline>" // Example: "I love building
small useful apps."
};

const instructorDetails = {
  name: "Rahul Dhangar",
  role: "Instructor",
  link: "https://github.com/rahuldhangar"
};
```

## 8b. Copy js/about.js

1. Open `Day-2-Tools-and-JavaScript\done\notes-board\js\about.js`.
2. Copy its whole content into a new file `js\about.js` in your own project.
3. Read it slowly. It uses the same ideas as `app.js`: it finds elements, creates new elements, and uses `textContent`. It also checks that a link starts with `https://`.

## 8c. Update `about.html`

1. In your `about.html`, find the `<section class="card">` that has the heading **Built by**. Replace the whole `<dl class="details"> ... </dl>` list and the hint paragraph under it with:

```
<!-- JavaScript fills this list from js/student-details.js -->
<dl id="student-details" class="details"></dl>
<p id="student-tagline" class="tagline"></p>
```

2. Find the last section (**Guided by**). Replace it with:

```
<section id="instructor-section" class="card">
  <h2>Guided by</h2>
  <!-- JavaScript fills this line from js/student-details.js -->
  <p id="instructor-line"></p>
</section>
```

3. At the very end of `<body>`, before `</body>`, add:

```
<script src="js/student-details.js"></script>
<script src="js/about.js"></script>
```

The order matters. `student-details.js` must come **before** `about.js`, because `about.js` uses the details.

4. Save, and open `about.html` in the browser.

**Check:** Your details appear. Empty values are hidden. The instructor line shows with a link. Now change one detail in `student-details.js`, save, refresh, and see it change on the page.

## Part 9: Your first server (15 minutes)

1. In the `Workshop` folder (next to `notes-board`), create a new folder named `hello-server`.
2. Inside it, create a file named `hello.js` and type:

```
// hello.js
// Our very first server, written with Node.js only (no extra packages).
//
// Run it:  node hello.js
// Open it: http://localhost:3000
// Stop it: press Ctrl + C in the terminal

const http = require("node:http");

const server = http.createServer(function (request, response) {
  // This function runs every time a browser asks for a page.
  console.log("Request received for:", request.url);

  response.writeHead(200, { "Content-Type": "text/plain; charset=utf-8" });
  response.end("Hello from my first Node.js server!");
});

server.listen(3000, function () {
  console.log("Server is running. Open http://localhost:3000 in your browser.");
});
```

3. In the terminal, go to the folder and run it:

```
cd hello-server
node hello.js
```

4. The terminal prints `Server is running...` and **stays busy**. This is correct. The server is waiting for requests.
5. Open your browser and go to `http://localhost:3000`. You see `Hello from my first Node.js server!`.
6. Look at the terminal. It printed `Request received for: /`. Your browser **requested**, and the server **responded**.
7. In the browser, try `http://localhost:3000/about`. The terminal prints a different path, but the answer is the same.
8. Go back to the terminal and press **Ctrl + C** to stop the server.

**Check:** You can explain in your own words: "The browser sent a request. The function in `hello.js` ran. The server sent a response."

## Part 10: Backup, quiz and homework

---

1. Copy your `Workshop` folder to a USB drive or to the place your instructor tells you.
  2. Do the 5-question quiz that your instructor shares.
  3. Read the **Homework for Day 3** at the end of the Day 2 Theory.
- 

## Final checklist

---

`node -v` shows version 24, and `node setup-check.js` shows four PASS lines.

My Notes Board adds and deletes notes, and shows an error for empty input.

My About page shows the details I chose, and the instructor line.

I ran `hello.js` and opened it in the browser.

I made a backup copy of my `Workshop` folder.

---

## Extra challenges (only if you finish early)

---

1. Show the date and time each note was created. Hint: `new Date().toLocaleString()`.
2. Add a button **Clear all notes** that empties the array.
3. Change `hello.js` so that the server says your name.
4. Add a `tagline` to your `student-details.js` and see it on the About page.
5. In the Console of your Notes Board page, type `notes` and press Enter. What do you see? Why?